# Lists and List Methods

**List**

A **list** is an ordered collection of data and is **mutable**, which means its elements can be changed after the list is created.

**Syntax**

list_name = [value1, value2, value3]

**Important Points**

- A list is an ordered collection.
- Lists are mutable.
- A list can contain different types of data.
- List elements are accessed using indexes.
- The index starts from 0.

**Example**

```
ages = [19, 26, 29]

print(ages)
```

**List with Different Data Types**

```
student = ["name", 20, 75.5, [1, 2], True]

print(student)
```

---

**2. Accessing List Elements**

**Accessing a Single Element by Index**

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])    # 10
print(numbers[3])    # 40
```

**Index Positions**

```
10   20   30   40   50
 0    1    2    3    4
```

---

**3. Accessing Elements Using Negative Indexing**

Negative indexing starts from the end of the list.

```
numbers = [10, 20, 30, 40, 50]

last_element = numbers[-1]

print(last_element)      # 50
print(numbers[-2])       # 40
```

**Negative Index Positions**

```
10   20   30   40   50
-5   -4   -3   -2   -1
```

---

**4. Iterating Over a List Using a for Loop**

A for loop can be used to access each element of a list.

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
for fruit in fruits:
    print(fruit)
```

**Output**

```
apple
banana
cherry
date
```

---

**5. Iterating Over a List Using a while Loop**

A list can also be traversed using a while loop.

```
fruits = ["apple", "banana", "cherry", "date"]

index = 0

while index < len(fruits):
    print(fruits[index])
    index += 1
```

Here, len() returns the number of elements in the list.

---

**List Methods**

Python provides several built-in methods for working with lists.

The PDF covers the following list methods:

1. append()
2. copy()
3. clear()
4. count()
5. extend()
6. index()
7. insert()
8. pop()
9. remove()
10. reverse()
11. sort()
12. min()
13. max()

---

**6. append()**

The append() method adds an element to the **end of the list**.

**Syntax**

```
list.append(element)
```

**Example**

```
numbers = [1, 2, 3]

numbers.append(4)

print(numbers)
```

**Output**

```
[1, 2, 3, 4]
```

---

**7. copy()**

The copy() method creates a **shallow copy** of the list.

**Example**

```
numbers = [1, 2, 3]

num_copy = numbers.copy()

print(num_copy)
```

**Output**

```
[1, 2, 3]
```

---

**8. clear()**

The clear() method removes **all elements** from the list.

**Example**

```
numbers = [1, 2, 3]

numbers.clear()

print(numbers)
```

**Output**

```
[]
```

---

**9. count()**

The count() method returns the number of occurrences of a specified element in the list.

**Example**

```
numbers = [1, 2, 2, 3]

count_of_two = numbers.count(2)

print(count_of_two)
```

**Output**

```
2
```

---

**10. extend()**

The extend() method adds elements of an iterable, such as another list, to the end of the current list.

**Example**

```
numbers = [1, 2, 3]

numbers.extend([4, 5])

print(numbers)
```

**Output**

```
[1, 2, 3, 4, 5]
```

---

**11. index()**

The index() method returns the index of the **first occurrence** of a specified element.

**Example**

```
numbers = [1, 2, 3, 2]

index_of_two = numbers.index(2)

print(index_of_two)
```
**Output**
1
The first 2 occurs at index 1.

---

**12. insert()**
The insert() method inserts an element at a specific position in the list.
**Syntax**
```
list.insert(index, element)
```
**Example**
```
numbers = [1, 2, 3]

numbers.insert(2, 4)

print(numbers)
```
**Output**
[1, 2, 4, 3]
Here, 4 is inserted at index 2.

---

**13. pop()**
The pop() method removes and returns an element at the specified index.
If no index is specified, it removes and returns the last element.
**Example**
```
numbers = [1, 2, 3]

last_element = numbers.pop()

print(last_element)
print(numbers)
```
**Output**
3
[1, 2]
**Removing an Element at a Specific Index**
```
numbers = [1, 2, 3, 4]

element = numbers.pop(1)

print(element)
print(numbers)
```
Output:
2
[1, 3, 4]

---

**14. remove()**

The remove() method removes the **first occurrence** of a specified element.

**Example**

```
numbers = [1, 2, 3, 2]

numbers.remove(2)

print(numbers)
```

**Output**

[1, 3, 2]

Only the first occurrence of 2 is removed.

---

**15. reverse()**

The reverse() method reverses the elements of the list.

**Example**

```
numbers = [1, 2, 3]

numbers.reverse()

print(numbers)
```

**Output**

[3, 2, 1]

---

**16. sort()**

The sort() method sorts the list in **ascending order by default**.

**Ascending Order**

```
numbers = [3, 1, 4, 2]

numbers.sort()

print(numbers)
```

**Output**

[1, 2, 3, 4]

**Descending Order**

Use reverse=True.

```
numbers = [3, 1, 4, 2]

numbers.sort(reverse=True)

print(numbers)
```

**Output**

[4, 3, 2, 1]

---

**17. min()**

The min() function returns the **smallest element** in the list.

**Example**

```
numbers = [3, 1, 4, 2]

minimum = min(numbers)
```

print(minimum)
**Output**
1

---

**18. max()**
The max() function returns the **largest element** in the list.
**Example**
numbers = [3, 1, 4, 2]

maximum = max(numbers)

print(maximum)
**Output**
4

---

**19. List Comprehension**
**List comprehension** provides a concise way to create lists in Python.
It is a shorthand for looping through an iterable and applying an expression to each item.
**Syntax**
new_list = [expression for item in iterable]

---

**Example 1 – Create a List of Squares**
squares = [x**2 for x in range(5)]

print(squares)
**Output**
[0, 1, 4, 9, 16]

---

**Example 2 – Create a List of Even Numbers**
even = [x for x in range(10) if x % 2 == 0]

print(even)
**Output**
[0, 2, 4, 6, 8]

---

**20. Nested Lists**
A **nested list** is a list that contains other lists as its elements.
**Example**
matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]
**Accessing an Element**
Nested-list elements can be accessed using multiple indexes.
print(matrix[1][2])
**Output**

6
Here:
matrix[1]  → [4, 5, 6]
matrix[1][2]  → 6

---

**21. Iterating Through a Nested List**
A nested list can be traversed using nested for loops.

```
matrix = [
   [1, 2, 3],
   [4, 5, 6],
   [7, 8, 9]
]

for row in matrix:
   for element in row:
      print(element, end=" ")

   print()
```

**Output**

```
1 2 3
4 5 6
7 8 9
```